

関数仕様書

プロジェクト: nextjsform (Next.js 16 / App Router)
対象: Route Handlers (`src/app/api/**/route.ts`)、型定義、DB 接続

1. 型定義 (`src/lib/types.ts`)

型名	フィールド (概要)
TaxRate	id, applicable_from, rate, description
ProductCategory	id, name, sort_order
Product	id?, product_code, product_name, category_id?, category_name?, manufacturer?, memo?, created_at?, updated_at?
ReceiptItem	id?, receipt_id?, product_code?, item_name, quantity, unit_price, amount, item_tax_rate, item_tax_amount
Receipt	id?, store_name, purchase_date, tax_rate_id, tax_rate?, tax_rate_description?, 金額系 (subtotal, tax_amount, total_amount), memo, タイムスタンプ, items?

2. DB 接続 (`src/lib/db.ts`)

項目	内容
エクスポート	mysql2 <code>createPool</code> のデフォルトエクスポート (<code>pool</code>)
環境変数	DB_HOST, DB_PORT, DB_USER, DB_PASSWORD, DB_NAME (未設定時は localhost / 3306 / root / 空 / sample_schema)
その他	<code>connectionLimit: 10</code> 、 <code>dateStrings: true</code> (日付のタイムゾーンずれ回避)

3. API — 税率 (`/api/tax-rates`)

GET `/api/tax-rates`

クエリなし: 税率マスタ全件 (`applicable_from` 昇順)。各要素は JSON 化時に `rate` を数値化。

HTTP	説明
200	<code>TaxRate</code> 相当のオブジェクトの配列
500	<code>{ "error": "DB エラー" }</code>

GET `/api/tax-rates?date=YYYY-MM-DD`

指定日以前で適用開始日が最新の1件を返す。

HTTP	説明
200	単一オブジェクト（税率1件）
404	{ "error": "該当する税率が見つかりません" }
500	{ "error": "DB エラー" }

4. API — 商品カテゴリ（`/api/product-categories`）

GET `/api/product-categories`

HTTP	説明
200	<code>id</code> , <code>name</code> , <code>sort_order</code> の一覧（ <code>sort_order</code> , <code>name</code> でソート）
500	<code>{ "error": "DB エラー" }</code>

5. API — 商品（`/api/products`）

GET `/api/products`

一覧は最大200件。オプション検索: `search`（コード・名称・メーカー部分一致）, `category_id`。単体は `code` で商品コード完全一致。

条件	HTTP	レスポンス
<code>?code=</code> で該当なし	404	<code>{ "error": "Not Found" }</code>
一覧 / 単一ヒット	200	オブジェクトまたは配列
DB 失敗	500	<code>{ "error": "DB エラー" }</code>

POST `/api/products`

ボディ: `Product` 形式（必須: `product_code`, `product_name`）。

HTTP	説明
201	<code>{ "id": <insertId> }</code>
400	必須チェックエラー文言
409	商品コード重複（ <code>ER_DUP_ENTRY</code> ）
500	DB エラー

6. API — 商品 ID（`/api/products/[id]`）

動的 `params` は Next.js の慣例どおり `Promise<{ id: string }>` を `await`。

GET

HTTP	説明
200	商品1件（カテゴリ名 JOIN 済み）
404	<code>{ "error": "Not Found" }</code>
500	DB エラー

PUT

ボディは POST と同様。必須: `product_code`, `product_name`。

HTTP	説明
200	<code>{ "id": <数値> }</code>
400 / 409 / 500	商品 POST と同様の意味

DELETE

HTTP	説明
200	<code>{ "ok": true }</code>
500	DB エラー

7. API — レシート（`/api/receipts`）

GET

一覧。税率は JOIN で `tax_rate` , `tax_rate_description` を付与。 `purchase_date` ・ `id` 降順。

HTTP	説明
200	<code>Receipt</code> 形状の配列（明細 <code>items</code> なし）
500	DB エラー

POST

ボディ: `Receipt` + `items` 任意。トランザクションで `receipts` 挿入後、`receipt_items` を一括 INSERT。

HTTP	説明
201	<code>{ "id": <receiptId> }</code>
500	ロールバック後 DB エラー

8. API — レシート ID（`/api/receipts/[id]`）

GET

ヘッダ + `receipt_items` を `items` 配列で返却。各明細の `item_tax_rate` は数値化。

HTTP	説明
200	明細付き <code>Receipt</code>
404	<code>{ "error": "Not Found" }</code>
500	DB エラー

PUT

ヘッダ更新後、当該 `receipt_id` の明細を全削除し、`items` を再挿入（トランザクション）。

HTTP	説明
200	<code>{ "id": <数値> }</code>
500	DB エラー

DELETE

HTTP	説明
200	<code>{ "ok": true }</code> （ <code>receipts</code> の該当行を削除）
500	DB エラー

9. エクスポート関数一覧 (Route Handler)

ファイル	関数 (HTTP)
api/tax-rates/route.ts	GET
api/product-categories/route.ts	GET
api/products/route.ts	GET , POST
api/products/[id]/route.ts	GET , PUT , DELETE
api/receipts/route.ts	GET , POST
api/receipts/[id]/route.ts	GET , PUT , DELETE

※ 本書はソースコード（`src/lib` , `src/app/api`）に基づく。仕様変更時は HTML を更新し、`npm run pdf:spec` で PDF を再生成すること。